



AutoClock — Development Document

Project	AutoClock — One-Click Daily Worklog & EOD Reporter
Document	Dev Doc v1.0 — the hands-on reference to build from
Audience	Keval, Yogesh, Ravi, Tejas (developers)
Hard rules	Zero cost · built for the real ~60 users
Reads from	AutoClock_PRD.md , AutoClock_ERD.md , AutoClock_WorkPlan.md

Keep this open while building. Every external call below has a copy-pasteable example so there are **no blockers** on the day.

1. Repository Structure

```
autoclock/
├── backend/
│   ├── server.js           # Express app entry
│   ├── db.js              # SQLite (WAL) connection + schema init
│   ├── auth/
│   │   ├── session.js     # login, session middleware
│   │   ├── rbac.js        # role-check middleware
│   │   ├── jiraOAuth.js   # Atlassian 3LO (pilot)
│   │   └── googleOAuth.js # Google OAuth (pilot)
│   ├── routes/
│   │   ├── entries.js     # EP-08..EP-11
│   │   ├── day.js        # EP-12 preview, EP-13 close
│   │   ├── projects.js   # EP-06, EP-07
│   │   ├── dashboard.js  # EP-14, EP-15
│   │   ├── ops.js        # EP-16..EP-18
│   │   └── admin.js      # EP-19..EP-22
│   ├── services/
│   │   ├── parser.js     # the free deterministic parser
│   │   ├── jira.js       # worklog write
│   │   ├── sheets.js     # Google Sheets append
│   │   ├── gmail.js      # Gmail draft create
│   │   └── crypto.js     # token encrypt/decrypt
│   ├── jobs/
│   │   └── compliance.js  # node-cron Fri/Mon
│   ├── .env              # secrets — NEVER committed
│   └── package.json
├── web/                  # React + Vite
│   ├── src/
│   │   ├── pages/        # Log, Preview, Dashboards, Admin
│   │   ├── components/
│   │   ├── api.js        # fetch wrapper
│   │   └── main.jsx
│   └── package.json
└── extension/           # Chrome MV3
```

```
| |─ manifest.json
| |─ background.js          # service worker + alarms
| |─ popup.html
| |─ popup.js
|─ README.md
```

2. Local Dev Setup (zero cost)

```
# 1. Prereqs (free): Node 18+, Git
node -v          # must be >= 18

# 2. Backend
cd backend
npm install express better-sqlite3 node-cron googleapis nodemailer dotenv cookie-parser
cp .env.example .env          # then fill in values
node server.js               # http://localhost:4000

# 3. Web app
cd ../web
npm install
npm run dev                  # http://localhost:5173

# 4. Extension
# Chrome → chrome://extensions → Developer mode ON → "Load unpacked" → select /extension
```

Every package above is MIT/free. No paid CLI, no paid service.

3. Environment Variables (**backend/.env**)

```
PORT=4000
SESSION_SECRET=<random-long-string>
TOKEN_ENC_KEY=<32-byte-key-for-aes-256>
# Hackathon demo - Jira Basic auth (one user). Use a CLASSIC token (ADR-11).
JIRA_BASE_URL=https://iksula.atlassian.net
JIRA_EMAIL=<demo-user-email>
JIRA_API_TOKEN=<fresh-3-day-classic-token>
# Only if the admin issues a SCOPED token instead of classic: set the cloudId.
# Then the base URL becomes https://api.atlassian.com/ex/jira/${JIRA_CLOUD_ID}
JIRA_CLOUD_ID=<from https://iksula.atlassian.net/_edge/tenant_info>
# Reader account - one broad "Browse Projects" account (Operations member),
# used ONLY to read all employees' worklogs for the dashboards (see §6.7):
JIRA_READER_EMAIL=<ops-member-email>
JIRA_READER_TOKEN=<ops-member-3-day-classic-token>
# Google sign-in (OIDC) - how users log in to AutoClock (ADR-10):
GOOGLE_OIDC_CLIENT_ID=<from Google Cloud Console>
GOOGLE_OIDC_CLIENT_SECRET=<...>
# Google API (Sheets + Gmail). DEMO: a refresh token for the demo account,
# captured before HackFest from an INTERNAL OAuth consent screen (see §6.4):
GOOGLE_CLIENT_ID=<from Google Cloud Console>
GOOGLE_CLIENT_SECRET=<...>
GOOGLE_REDIRECT_URI=http://localhost:4000/api/auth/google/callback
GOOGLE_DEMO_REFRESH_TOKEN=<demo-account-refresh-token>
# Atlassian OAuth (pilot - per-user):
ATLASSIAN_CLIENT_ID=<from developer.atlassian.com>
ATLASSIAN_CLIENT_SECRET=<...>
ATLASSIAN_REDIRECT_URI=http://localhost:4000/api/auth/jira/callback
```

.env is in **.gitignore**. Commit only **.env.example** with empty values.

4. The Free Parser — Exact Spec ([services/parser.js](#))

No paid AI. Pure JavaScript. Three jobs:

4.1 Duration → minutes

```
function parseDuration(raw) {
  // handles "30m", "1h", "1.5h", "1h 30m", "90", "90 min", "2 hrs"
  const s = String(raw).toLowerCase().trim();
  let mins = 0;
  const h = s.match(/(\d+(?:\.\d+)?)\s*(?:h|hr|hrs|hour|hours)/);
  const m = s.match(/(\d+)\s*(?:m|min|mins|minute|minutes)/);
  if (h) mins += Math.round(parseFloat(h[1]) * 60);
  if (m) mins += parseInt(m[1], 10);
  if (!h && !m) { const n = s.match(/^(?!\d+)$/); if (n) mins = parseInt(n[1], 10); }
  return mins; // integer minutes
}
```

4.2 Description tidy

```
const SHORTHAND = { scrum: "Daily Scrum", standup: "Stand-up",
  "tc": "test cases", grooming: "Backlog Grooming", uat: "UAT" };
function tidy(desc) {
  let d = desc.trim().replace(/\s+/g, " ");
  for (const [k, v] of Object.entries(SHORTHAND))
    d = d.replace(new RegExp(`\\b${k}\\b`, "gi"), v);
  return d.charAt(0).toUpperCase() + d.slice(1);
}
```

4.3 Group by ticket (for the preview + Jira write)

```
function groupByTicket(entries) {
  const map = {};
  for (const e of entries) {
    const key = e.jira_key;
    if (!map[key]) map[key] = { jira_key: key, minutes: 0, lines: [] };
    map[key].minutes += e.duration_minutes;
    map[key].lines.push(tidy(e.description));
  }
  return Object.values(map); // [{ jira_key, minutes, lines }]
}
```

4.4 Validation (run before "Close My Day" is allowed). Anything that **blocks** Confirm must be caught here — never let a bad value reach Jira/Sheets:

- `duration_minutes` must be **> 0** (`parseDuration` returns 0 for junk input — reject it).
- `slot_end` must be **after** `slot_start`.
- Each entry must have a `jira_task_id` and a non-empty `description`.
- Total minutes ≤ 1440 (24h) — else block with a message.
- Flag overlapping slots — warn in the preview (do not auto-block).
- A duration that does not match its slot length → warn (do not block).

5. Backend Endpoints — implementation notes

- Use Express routers per file (§1). Every route except `/api/auth/*` runs `requireSession` then `requireRole(...)`.
- **EP-12 `/api/day/preview`** — load the day's `worklog_entries`, run `groupByTicket`, return `{ groups, totalMinutes, warnings }`.
- **EP-13 `/api/day/close`** — **idempotent** per `(user_id, work_date)`. For each (entry, system), check `external_writes` (§6.8): skip rows already `synced`, (re)try `pending/failed` ones. Wrap each call in try/catch so one failure never blocks the others; record the `external_id` and status. A double-click or retry therefore cannot create a duplicate.
- Return the per-system result object (ERD §6 example).

6. Integration How-Tos (copy-paste ready)

6.1 Jira — create a worklog (`services/jira.js`)

Base URL depends on the token type (ADR-11). A **classic** token → `JIRA_BASE_URL` (`https://{site}.atlassian.net`). A **scoped** token → `https://api.atlassian.com/ex/jira/${JIRA_CLOUD_ID}`. Request a **classic** token so the code below works as-is.

```
const JIRA_API = process.env.JIRA_CLOUD_ID
  ? `https://api.atlassian.com/ex/jira/${process.env.JIRA_CLOUD_ID}` // scoped token
  : process.env.JIRA_BASE_URL; // classic token
// Hackathon: Basic auth with email + API token.
async function createWorklog(jiraKey, minutes, description, startIso) {
  const auth = Buffer.from(`${process.env.JIRA_EMAIL}:${process.env.JIRA_API_TOKEN}`)
    .toString("base64");
  const body = {
    timeSpentSeconds: minutes * 60,
    started: startIso, // e.g. "2026-05-22T14:30:00.000+0530" (IST offset!)
    comment: {
      type: "doc", version: 1,
      content: [{ type: "paragraph",
        content: [{ type: "text", text: description }] }]
    }
  };
  const res = await fetch(
    `${JIRA_API}/rest/api/3/issue/${jiraKey}/worklog`,
    { method: "POST",
      headers: { "Authorization": `Basic ${auth}`,
        "Content-Type": "application/json" },
      body: JSON.stringify(body) });
  if (!res.ok) throw new Error(`Jira ${res.status}: ${await res.text()}`);
  return (await res.json()).id; // jira_worklog_id
}
```

Notes: the worklog is attributed to the token owner (ERD ADR-01). `started` MUST carry the `+0530` IST offset. Required Jira permission: *Work on issues*.

6.2 IST → the `started` string

```
function toJiraStarted(workDate, slotStart) { // "2026-05-22", "14:30"
  return `${workDate}T${slotStart}:00.000+0530`;
}
```

6.3 Google Sheets — append a row (`services/sheets.js`)

```
const { google } = require("googleapis");
async function appendRow(authClient, spreadsheetId, row) {
  const sheets = google.sheets({ version: "v4", auth: authClient });
  await sheets.spreadsheets.values.append({
    spreadsheetId,
    range: "May 2026!A:C", // match the REAL sheet (get a copy first!)
    valueInputOption: "USER_ENTERED",
    requestBody: { values: [row] } // e.g. [["14:30-16:00", "1h 30m", "UOM test cases"]]
  });
}
```

Scope: <https://www.googleapis.com/auth/spreadsheets>.

Sheet contract — freeze before Friday (OQ-3). `values.append` searches the given `range` for a *logical table* and appends **after its last row** — `valueInputOption` does **not** set the start cell. The `range: "May 2026!A:C"` above is a **placeholder**. Pin the real contract: exact tab name, the data-table A1 range, column order, and **row-per-ticket** (one row per ticket group). Test against a **copy** of the real sheet — formulas / merged cells / hidden columns will misplace rows. Per-user `sheet_id + sheet_range` come from `users` (ERD §14.3).

6.4 Gmail — create the EOD draft (`services/gmail.js`)

```
const { google } = require("googleapis");
function buildMime(to, subject, html) {
  const msg =
    `To: ${to}\r\nSubject: ${subject}\r\n` +
    `Content-Type: text/html; charset=UTF-8\r\n\r\n${html}`;
  return Buffer.from(msg).toString("base64")
    .replace(/\+/g, "-").replace(/\//g, "_").replace(/>=+$/, "");
}
async function createDraft(authClient, to, subject, html) {
  const gmail = google.gmail({ version: "v1", auth: authClient });
  const res = await gmail.users.drafts.create({
    userId: "me",
    requestBody: { message: { raw: buildMime(to, subject, html) } }
  });
  return res.data.id; // gmail_draft_id
}
```

Scope: <https://www.googleapis.com/auth/gmail.compose> — this is a **restricted** Google scope. To avoid the multi-week verification process, the **OAuth consent screen must be set to "Internal"** and the Cloud project owned by the Iksula Workspace org (then an internal app is exempt from brand/restricted-scope verification). This is a deliberate setup task — do it **before HackFest** (Work Plan P-4), not during.

6.5 Atlassian OAuth 2.0 (3LO) — per-user connect (pilot)

1. Register an app at developer.atlassian.com → Authorization → OAuth 2.0 (3LO). Free.
2. Authorization URL (redirect the user here):

```
https://auth.atlassian.com/authorize?audience=api.atlassian.com
&client_id=<ATLASSIAN_CLIENT_ID>
&scope=read:jira-work%20write:jira-work%20offline_access
&redirect_uri=<ATLASSIAN_REDIRECT_URI>
&state=<per-user-random>&response_type=code&prompt=consent
```

1. Callback receives `?code=` → POST to <https://auth.atlassian.com/oauth/token> with `grant_type=authorization_code, client_id, client_secret, code, redirect_uri`.

2. Store `access_token + refresh_token` (encrypted) in `user_connections` (TB-06).
3. Refresh with `grant_type=refresh_token`.

⚠ **Rotating refresh tokens.** Each refresh call returns a **NEW** `refresh_token` and **invalidates the old one**. You **must** overwrite the stored refresh token (in the same transaction as the new access token) on every refresh — if you keep reusing the original, Jira access dies. Also: a refresh token expires after ~90 days idle → the user must re-connect.

6.6 Token encryption (`services/crypto.js`)

Use Node's built-in `crypto`, **AES-256-GCM**, `key = TOKEN_ENC_KEY` (env). Store each token as one string `iv:authTag:ciphertext` (each part base64) — never store ciphertext without its IV and auth tag, or decryption fails. `encrypt()` generates a random 12-byte IV per call; `decrypt()` splits on `:`, verifies the auth tag, throws on tamper. No key rotation in M0 — if the key is lost, users re-connect. Never store a token in plain text (ERD §10 / §14.4).

6.7 Jira — read ALL worklogs for the dashboards (`services/worklogSync.js`)

The Operations & Management dashboards need every employee's hours, incl. history. Pull them with the **reader account** (`JIRA_READER_*` — the Ops member's broad "Browse Projects" account). **No Clockwork API** — this is the same native Jira worklog data Clockwork displays.

```
// Step 1 - get IDs of all worklogs changed since a date (paginated)
async function worklogIdsSince(sinceMillis) {
  const auth = Buffer.from(`${process.env.JIRA_READER_EMAIL}:${process.env.JIRA_READER_TOKEN}`)
    .toString("base64");
  let ids = [], since = sinceMillis, lastPage = false;
  while (!lastPage) {
    const r = await fetch(`${process.env.JIRA_BASE_URL}/rest/api/3/worklog/updated?since=${since}`,
      { headers: { Authorization: `Basic ${auth}` } });
    const j = await r.json();
    ids.push(...j.values.map(v => v.worklogId));
    lastPage = j.lastPage; since = j.until;
  }
  return ids;
}

// Step 2 - fetch full worklog objects (POST, max 1000 ids per call)
async function worklogDetails(ids) {
  const auth = Buffer.from(`${process.env.JIRA_READER_EMAIL}:${process.env.JIRA_READER_TOKEN}`)
    .toString("base64");
  const out = [];
  for (let i = 0; i < ids.length; i += 1000) {
    const r = await fetch(`${process.env.JIRA_BASE_URL}/rest/api/3/worklog/list`,
      { method: "POST",
        headers: { Authorization: `Basic ${auth}`, "Content-Type": "application/json" },
        body: JSON.stringify({ ids: ids.slice(i, i + 1000) }) });
    out.push(...await r.json());
  }
  return out; // each: { author, timeSpentSeconds, started, issueId, comment }
}
```

Then filter by `started >= since`, aggregate by `author.accountId × day/week`, and cache for EP-23. A worklog is returned only if the reader account has **Browse Projects** on its project — that is why one broad account is used. Note: `worklog/updated` is paginated and `worklog/list` accepts max 1000 IDs per call (the loop above handles both); worklogs with visibility restrictions may not appear, so do not promise *exhaustive* history.

6.8 Idempotent "Close My Day" (`services/sync.js`) — the `external_writes` ledger

"Close My Day" writes to three side-effectful systems. Without a guard, a double-click or a retry after a partial failure **duplicates** worklogs. The `external_writes` table (ERD TB-13) makes the sync safe and retryable.

```
// For each (entry, system) the close handler does:
async function syncOne(entry, system, doWrite) {
  // 1. find or create the ledger row for this (entry, system)
  let row = db.prepare(
    "SELECT * FROM external_writes WHERE worklog_entry_id=? AND system=?"
  ).get(entry.id, system);
  if (row && row.status === "synced") return row; // already done — skip
  if (!row) {
    db.prepare(`INSERT INTO external_writes
      (worklog_entry_id,user_id,work_date,system,status,attempt_count,created_at,updated_at)
      VALUES (?, ?, ?, ?, 'pending', 0, datetime('now'), datetime('now'))`)
      .run(entry.id, entry.user_id, entry.work_date, system);
    row = db.prepare("SELECT * FROM external_writes WHERE worklog_entry_id=? AND system=?")
      .get(entry.id, system);
  }
  // 2. attempt the external write
  try {
    const externalId = await doWrite(); // returns jira worklog id / etc.
    db.prepare(`UPDATE external_writes SET status='synced', external_id=?,
      attempt_count=attempt_count+1, updated_at=datetime('now') WHERE id=?`)
      .run(externalId, row.id);
  } catch (e) {
    db.prepare(`UPDATE external_writes SET status='failed', last_error=?,
      attempt_count=attempt_count+1, updated_at=datetime('now') WHERE id=?`)
      .run(String(e), row.id);
  }
  return db.prepare("SELECT * FROM external_writes WHERE id=?").get(row.id);
}
```

Rules: `POST /api/day/close` is keyed on `(user_id, work_date)`; it only ever touches rows not already `synced`; retrying re-runs **only** failed systems. This is what makes FR-08 (retry) and FR-22 (idempotency) real.

7. Chrome Extension (MV3)

`manifest.json`

```
{
  "manifest_version": 3,
  "name": "AutoClock",
  "version": "1.0.0",
  "icons": { "16": "icon16.png", "48": "icon48.png", "128": "icon128.png" },
  "action": { "default_popup": "popup.html" },
  "background": { "service_worker": "background.js" },
  "permissions": ["alarms", "notifications", "storage"],
  "host_permissions": ["http://localhost:4000/*", "https://<your-host>/*"]
}
```

`background.js` — register the listener at the TOP LEVEL (MV3 rule):

```
chrome.runtime.onInstalled.addListener(() => {
  chrome.alarms.create("reminder", { periodInMinutes: 120 }); // gentle default
});
chrome.alarms.onAlarm.addListener((alarm) => { // top-level — required
  if (alarm.name === "reminder") {
    chrome.notifications.create({
      type: "basic", iconUrl: "icon48.png",
      title: "AutoClock", message: "Log your recent work? (20 seconds)"
    });
  }
});
```

```
}  
});
```

Rules: service workers sleep when idle — keep state in `chrome.storage.local`, never global variables; never `setInterval`. The popup is a normal REST client of the backend.

Offline — M1, not M0. For the hackathon the popup needs connectivity to save; if offline it shows a clear "offline — not saved" state. A true offline queue (local entry buffer in `chrome.storage`, conflict handling, reconciliation on reconnect) is an **M1/pilot** item (ERD §14.6) — do not attempt it during the 18-hour build.

8. Database (`db.js`)

```
const Database = require("better-sqlite3");  
const db = new Database("autoclock.db");  
db.pragma("journal_mode = WAL"); // concurrency for ~60 users  
// run schema.sql on first boot - tables TB-01..TB-13 from ERD §5
```

Keep the schema DDL in `backend/schema.sql` matching ERD §5 exactly (column names, types, FKs) — including **TB-13 `external_writes`** (the idempotent sync ledger, §6.8) and the `users.google_sub` / `users.onboarding_status` columns (login, §8.1). Add a unique index on `external_writes(worklog_entry_id, system)`. Seed with ERD §5.2 data on first run.

8.1 Login — Google Workspace sign-in (ADR-10)

AutoClock sign-in (`EP-01`) is **Google OIDC**, not a password: redirect to Google, the user signs in with their Iksula Google account, the callback gives an ID token, you verify it and read the `sub` claim. Match or create the `users` row by `google_sub`; set `onboarding_status`. This is separate from the per-user *API* tokens for Jira/Google writes (those are connected later, in `user_connections`). No password hashing, no reset flow — zero cost, fits the org.

9. Coding Conventions

- **Language:** JavaScript (ES2022), `async/await` everywhere — no callback nesting.
- **Naming:** `camelCase` for JS, `snake_case` for DB columns (matches the ERD).
- **Errors:** every external call wrapped in try/catch; never swallow — log + return a clear message.
- **No secrets in code.** Ever. `.env` only.
- **Small files** — one router/service per concern (§1).
- **AI-assisted coding is fine** (Keval's strength) — but every generated block must be read, understood, and tested before commit.

10. Git Workflow

- `main` is always demoable. Branch per feature: `feat/entries`, `feat/jira-sync`, `feat/dashboard`.
 - Commit small and often with clear messages.
 - Merge to `main` only when the feature works end-to-end (Work Plan §8 — Definition of Done).
 - The API contract (ERD §6) is frozen at Hr 1 — changing it needs a team OK.
-

11. Testing

- **Parser** — unit-test `parseDuration` against: 30m, 1h, 1.5h, 1h 30m, 90, 90 min, 2 hrs.
- **Integrations** — test each (`jira.js`, `sheets.js`, `gmail.js`) standalone with one real call before wiring into `/api/day/close`.
- **End-to-end** — Yogesh runs the full employee flow on the **demo laptop**, happy path + one error path (e.g., a bad ticket → 403), every phase.
- **Sheet safety** — test `appendRow` against a **copy** of the real sheet, never the live one, until verified.

12. Deployment (self-host, zero cost)

```
# On the Iksula internal server (Tejas):
git clone <repo> && cd autoclock/backend
npm ci --omit=dev
npm install -g pm2          # free process manager
pm2 start server.js --name autoclock
pm2 startup && pm2 save      # survive reboots
```

- Build the web app: `cd web && npm run build` → serve `web/dist` from Express.
- HTTPS via the internal certificate or Let's Encrypt (free).
- Nightly backup: `cp autoclock.db backups/autoclock-$(date +%F).db` via cron.
- Extension: force-install via the Google Workspace Admin Console (pilot).

13. Demo-Day Runbook

1. Backend running under PM2 on the demo laptop; web app served; extension loaded unpacked.
2. Jira token fresh and pre-tested (one successful worklog already posted that day).
3. Three browser tabs open: Clockwork on a ticket, the Google Sheet, Gmail.
4. Run the 60-second demo (PRD §7 Flow B / brainstorm §13.2).
5. **Backup video** ready to play if the live demo fails.
6. Keep the environment frozen after Hr 16 — no new code.

14. Troubleshooting

Symptom	Likely cause	Fix
Jira POST → 401	Bad/expired API token	Generate a fresh token; check <code>email:token</code> Base64
Jira POST → 403	No <i>Work on issues</i> permission on that ticket	Use a ticket the demo user can log on; check project permissions
Jira POST → 429	Rate limited	Exponential backoff + jitter; space the batch
Worklog on wrong day in Clockwork	Missing/incorrect IST offset	<code>started</code> must end with <code>+0530</code>
Sheet data in wrong columns	<code>range</code> mismatch	Match <code>range</code> to the real sheet; test on a copy

Gmail draft not in user's inbox	Wrong account / scope	<code>userId: "me"; scope gmail.compose;</code> per-user token
Extension reminder fires once then stops	Listener registered inside a callback	Register <code>onAlarm</code> at the top level of <code>background.js</code>
<code>SQLITE_BUSY</code>	Concurrent writes	WAL mode enabled in <code>db.js</code> ; keep transactions short
Jira call → 404 / "not found" with a scoped token	Wrong base URL	Scoped tokens need <code>https://api.atlassian.com/ex/jira/{cloudId}</code> — set <code>JIRA_CLOUD_ID</code> , or use a classic token (ADR-11)
Pilot user randomly loses Jira access	Stale refresh token	Rotating refresh tokens — store the NEW <code>refresh_token</code> returned by every refresh (§6.5)
Duplicate worklogs after a retry / double-click	No idempotency guard	Use the <code>external_writes</code> ledger — skip rows already <code>synced</code> (§6.8)
Google OAuth shows "unverified app" warning	Consent screen is "External"	Set the OAuth consent screen to Internal , project owned by the Iksula Workspace org (§6.4)
Extension popup blank	Service worker error	Check <code>chrome://extensions</code> → service worker console

Build order suggestion: §4 parser → §8 DB → §5 endpoints → §6 integrations → §7 extension. Follow the Work Plan phases.

End of Dev Doc v1.0 — AutoClock, Iksula HackFest 2026.
